PSG College of Technology
Department of Computer Applications
26MX11 – Applied Mathematics for Computer Science
Unit 4: Optimization Techniques — Lecture Notes

Gradient Descent Made Simple

Batch, Stochastic and Mini-batch Gradient Descent, with Worked Examples

How to use these notes

These notes start from zero. **Part A** explains gradient descent on simple functions, where every step can be done with a calculator. **Part B** applies the same rule to a loss built from data; this is where batch, stochastic and mini-batch gradient descent come from. All three are worked on one tiny data set of four points, so you can check every number by hand. **Part C** shows that this is exactly what happens when a machine-learning model or a deep neural network is "trained". Practice problems with answers are at the end.

Symbol	Meaning
α	learning rate (step size); many books write η
$f'(x)$	derivative of f: the slope at x
∇f	gradient: the list of slopes in every direction
w, b	weights (parameters) of a model: the knobs that training adjusts
$J(w)$	loss: the average error of the model over the data
$g_i(w)$	the gradient contributed by data point i alone
n, B	number of data points; mini-batch size

Notation used in these notes

Part A. The mathematics: rolling downhill

1. The idea: walking down a hill in fog

Imagine you are standing on a hillside in thick fog and want to reach the lowest point of the valley. You cannot see the valley. All you can do is feel the ground under your feet: which way it slopes down, and how steeply. A sensible plan is: feel the slope, take a small step downhill, and repeat. When the ground under your feet is flat, you are at the bottom.

Gradient descent is exactly this plan written in mathematics.

In the hill story	In mathematics
where you stand	the current value of x (in ML: the weights w)
height of the ground	the function value f(x) (in ML: the loss J)
slope under your feet	the derivative f′(x) (with many variables: the gradient ∇f)
length of your step	the learning rate α
flat ground at the bottom	f′(x) = 0: a minimum

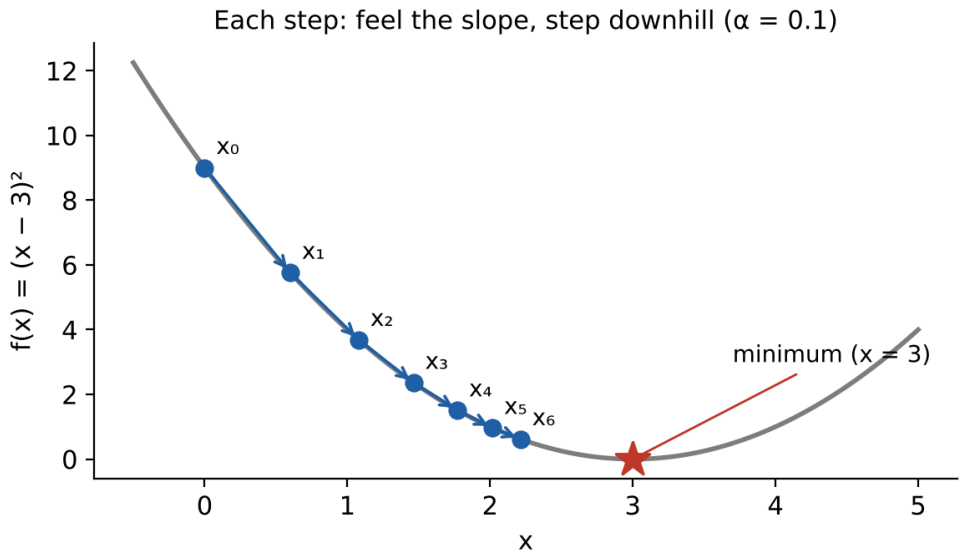


Figure 1. Gradient descent on f(x) = (x − 3)^2 from x = 0 with α = 0.1. The steps get shorter as the ground flattens.

2. The slope tells us which way is downhill

For a function of one variable, the derivative f′(x) is the slope: how much f changes when x increases a little. Two facts are all we need.

- If f′(x) is **negative**, f goes down as x increases: downhill is to the right, so x should **increase**.
- If f′(x) is **positive**, downhill is to the left, so x should **decrease**.
- The larger |f′(x)|, the steeper the ground. f′(x) = 0 means flat ground.

In both cases we move x in the direction **opposite** to the sign of the slope. That is where the minus sign in the update rule comes from. For $f(x) = (x - 3)^2$ the slope is $f'(x) = 2(x - 3)$:

x	0	2	3	4	6
slope f′(x)	−6	−2	0	2	6
what it says	steep: go right	gentle: go right	flat: minimum	gentle: go left	steep: go left

3. The update rule

$$x_{new} = x_{old} - \alpha \times f'(x_{old})$$

In plain words: new position = old position − (step size) × (slope). Steep ground gives a big step, gentle ground a small step and flat ground no step at all, so the method slows down by itself as it nears the bottom.

Worked Example 1. One variable: $f(x) = (x - 3)^2$, start $x_0 = 0$, $\alpha = 0.1$

The rule becomes $x_{new} = x - 0.1 \times 2(x - 3) = x - 0.2(x - 3)$.

Step k	x_k	slope $2(x_k - 3)$	move $-\alpha \times$ slope	x_{k+1}	$f(x_{k+1})$
0	0	−6	+0.6	0.6	5.76
1	0.6	−4.8	+0.48	1.08	3.6864
2	1.08	−3.84	+0.384	1.464	2.3593
3	1.464	−3.072	+0.3072	1.7712	1.5100
4	1.7712	−2.4576	+0.2458	2.0170	0.9664

Three things to notice: (i) every step goes right, because the slope is negative; (ii) the steps get shorter (0.6, 0.48, 0.384, …) because the ground gets flatter; (iii) the distance to the minimum shrinks by the same factor every step: $3 - x_{k+1} = 0.8\,(3 - x_k)$, so the distances are 3, 2.4, 1.92, 1.536, … We never land exactly on 3, but we get as close as we like.

4. Choosing the learning rate

Worked Example 2. The same function with five learning rates ($x_0 = 0$)

α	x_1	x_2	x_3	x_4	x_5	What happens
0.01	0.06	0.1188	0.1764	0.2329	0.2882	right direction, far too slow
0.1	0.6	1.08	1.464	1.7712	2.0170	steady progress
0.5	3	3	3	3	3	lands on 3 in one step (true only for this f)
1.0	6	0	6	0	6	bounces between 0 and 6 for ever
1.1	6.6	−1.32	8.184	−3.221	10.465	overshoots more each time: diverges

Why? For this f, $x_{new} - 3 = (1 - 2\alpha)(x_{old} - 3)$: every step multiplies the distance to the minimum by $(1 - 2\alpha)$. The distance shrinks only if $|1 - 2\alpha| < 1$, that is, $0 < \alpha < 1$. At $\alpha = 1$ the factor is −1 (a bounce); at $\alpha = 1.1$ it is −1.2 (the distance grows).

In plain words: too small is safe but slow; too large overshoots the valley and can climb up the other side. In practice we try a value such as 0.1 or 0.01, watch whether the loss goes down, and divide α by 10 if the loss goes up or jumps around.

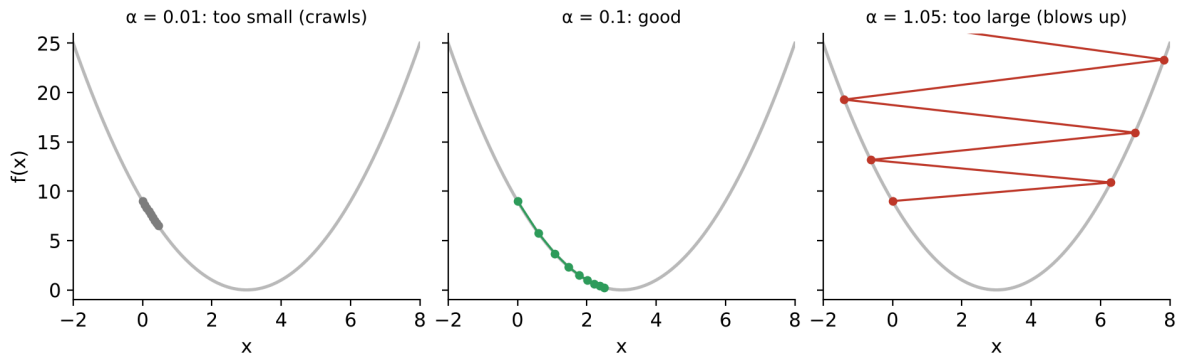


Figure 2. Eight steps with a learning rate that is too small, good, and too large.

5. Two or more variables: the gradient

Real problems have many variables. For a function f(w, b) we find the slope in each direction separately (the partial derivatives) and collect them in a vector called the **gradient**, $\nabla f = (\partial f/\partial w, \partial f/\partial b)$. The gradient points uphill (the direction of steepest increase), so we step the opposite way, updating all the variables at the same time:

$$w_{new} = w - \alpha\, \partial f/\partial w, \qquad b_{new} = b - \alpha\, \partial f/\partial b$$

Worked Example 3. Two variables: $f(w, b) = (w - 2)^2 + 2(b + 1)^2$, start (0, 0), $\alpha = 0.1$

The gradient is $\nabla f = (2(w - 2), 4(b + 1))$. The minimum is at $(2, -1)$.

Step	(w, b)	gradient ∇f	f(w, b)
0	(0, 0)	(−4, 4)	6
1	(0.4, −0.4)	(−3.2, 2.4)	3.28
2	(0.72, −0.64)	(−2.56, 1.44)	1.8976
3	(0.976, −0.784)	(−2.048, 0.864)	1.1419

Both coordinates move towards $(2, -1)$ together. b moves faster at first because f is steeper in the b direction (the factor 2 in front of $(b + 1)^2$).

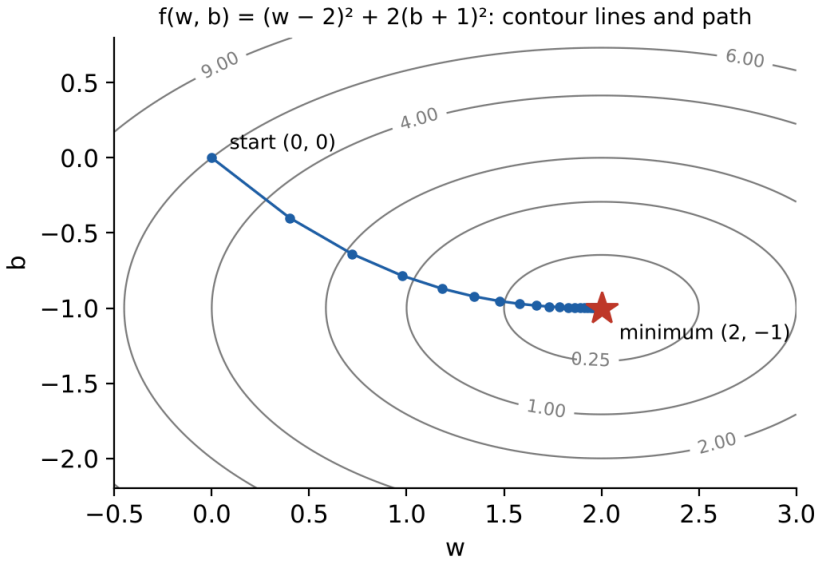


Figure 3. Contour lines of f(w, b) (lines of equal height, like a map) and the gradient-descent path.

6. When do we stop?

Common stopping rules: stop when the slope is almost zero (for example $|f'(x)| < 0.001$), when f stops decreasing noticeably, or after a fixed number of steps. In machine learning the last is the usual choice: we fix the number of epochs (passes through the data).

Part B. From a formula to data: batch, stochastic and mini-batch

7. A loss function built from data

In machine learning nobody hands us the function to minimise; we build it from data. Take four data points. To give them a story, let x be the size of a file in MB and y the time in seconds that it took to upload.

Point i	1	2	3	4
input x_i	1	2	3	4
target y_i	3	5	5	8

We want a rule $\hat{y} = wx$ (a straight line through the origin) that predicts y from x. The single number w is the knob that we turn. For point i the **error** is $e_i = \hat{y}_i - y_i = wx_i - y_i$, and the loss is the **mean squared error** (MSE), the average of the squared errors:

$$J(w) = (1/n)\,\Sigma\,(wx_i - y_i)^2 = \tfrac{1}{4}[(w-3)^2 + (2w-5)^2 + (3w-5)^2 + (4w-8)^2]$$

w	0	1	2	3
J(w)	30.75	8.25	0.75	8.25

Expanding the squares gives $J(w) = 7.5w^2 - 30w + 30.75 = 7.5(w-2)^2 + 0.75$. So J is a bowl with its lowest point at w = 2, where J = 0.75 (not 0, because no line through the origin passes through all four points). Gradient descent should therefore take w to 2. Knowing the answer in advance lets us check every method.

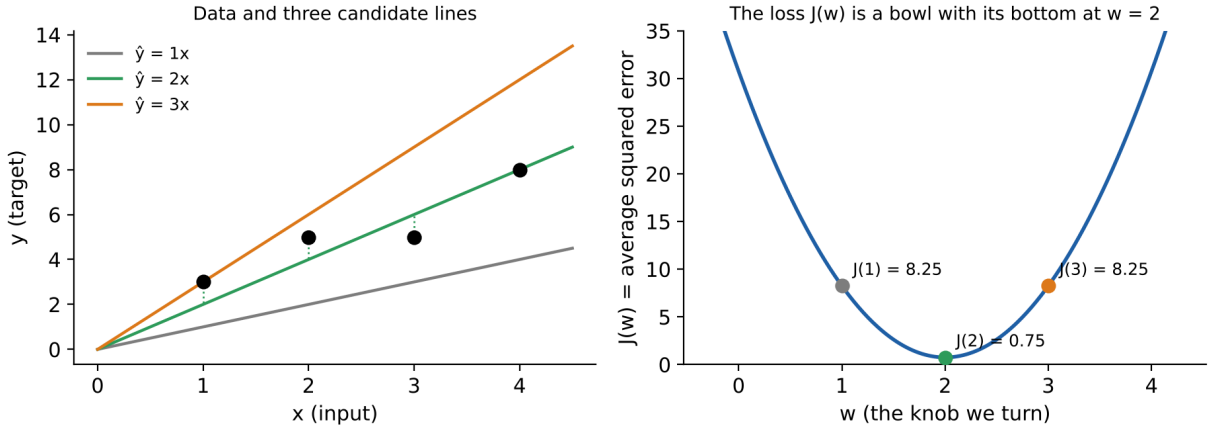


Figure 4. Left: the data and three candidate lines. Right: the loss J(w); the best line is $\hat{y} = 2x$.

8. The key fact: the slope of an average is the average of the slopes

The derivative of one squared error $(wx_i - y_i)^2$ is $2x_i(wx_i - y_i)$. Call it $g_i(w)$, the gradient contributed by point i:

$$g_i(w) = 2 \times (\text{error}) \times (\text{input}) = 2(wx_i - y_i)\,x_i$$

Because J is the average of the four squared errors, its derivative is the average of the four g_i:

$$J'(w) = (1/n)\,[g_1(w) + g_2(w) + \ldots + g_n(w)]$$

Point i	error $e_i = 0 \times x_i - y_i$	$g_i(0) = 2e_ix_i$
1	−3	−6
2	−5	−20
3	−5	−30
4	−8	−64
average		−120/4 = −30 = J′(0)

The four votes at w = 0

> **In plain words:** think of each data point as casting a **vote** on which way w should move and how strongly. The full gradient is the average vote of all the points. Batch, stochastic and mini-batch gradient descent differ in only one thing: how many votes we count before taking a step.

Method	Votes counted per step	Steps per pass through the data
Batch gradient descent	all n points	1
Stochastic gradient descent (SGD)	1 point	n
Mini-batch gradient descent	B points (for example 32)	n/B

One pass through all the training data is called an **epoch**. Each update of w is called an **iteration** (or step).

9. Batch gradient descent: count every vote

Worked Example 4. Batch gradient descent, $\alpha = 0.05$, $w_0 = 0$

Each step uses the average of all four votes: $w_{new} = w - 0.05 \times J'(w)$. (Here $J'(w) = 15(w - 2)$, which you can check by averaging the four g_i.)

$0 - 0.05 \times (-30)$

$1.5 - 0.05 \times (-7.5)$

Epoch	w before	J′(w): average of 4 votes	w after	J(w) after
1	0	−30	1.5	2.625
2	1.5	−7.5	1.875	0.867
3	1.875	−1.875	1.96875	0.757
4	1.96875	−0.46875	1.99219	0.7505

Smooth and steady: the distance to 2 is multiplied by 0.25 at every step (2, 0.5, 0.125, 0.031, …). The catch is that **every step needs every data point**. With 4 points that costs nothing; with 10 million images it means reading the whole data set to take a single step.

10. Stochastic gradient descent (SGD): one vote at a time

SGD takes a step after **every single point**, using only that point's vote: $w_{new} = w - \alpha\, g_i(w)$. "Stochastic" means random: in practice the points are visited in a random order, reshuffled every epoch. Here we use the order 1, 2, 3, 4 so that you can check the numbers.

Worked Example 5. Stochastic gradient descent, $\alpha = 0.05$, $w_0 = 0$, first epoch

Update	Point used	w before	vote $g_i(w) = 2x_i(wx_i - y_i)$	w after	J(w) after
1	1: (1, 3)	0	$2 \cdot 1 \cdot (0 - 3) = -6$	0.3	22.43
2	2: (2, 5)	0.3	$2 \cdot 2 \cdot (0.6 - 5) = -17.6$	1.18	5.79
3	3: (3, 5)	1.18	$2 \cdot 3 \cdot (3.54 - 5) = -8.76$	1.618	1.84
4	4: (4, 8)	1.618	$2 \cdot 4 \cdot (6.472 - 8) = -12.224$	2.2292	1.14

After one epoch SGD has made **4 updates** and reached w = 2.23 (J = 1.14). Batch gradient descent, after the same amount of arithmetic (4 votes computed), had made one update and reached only w = 1.5 (J = 2.63).

But look at what happens next. In the second epoch w goes 2.31, 2.38, 1.74, 2.16; in the third, 2.24, 2.34, 1.73, 2.16. With a constant α, SGD **does not settle** at 2: it keeps jittering between about 1.7 and 2.4. Section 13 explains why.

11. Mini-batch gradient descent: a few votes at a time

Mini-batch gradient descent averages the votes of a small group of B points (a mini-batch) for each step. With B = 2 we split the data into mini-batch 1 = points {1, 2} and mini-batch 2 = points {3, 4}.

Worked Example 6. Mini-batch gradient descent, B = 2, $\alpha = 0.05$, $w_0 = 0$, first epoch

Update	Mini-batch	w before	the two votes	average	w after	J(w) after
1	{1, 2}	0	−6, −20	−13	0.65	14.42
2	{3, 4}	0.65	$2 \cdot 3 \cdot (1.95 - 5) = -18.3$; $2 \cdot 4 \cdot (2.6 - 8) = -43.2$	−30.75	2.1875	1.01

In the second epoch w goes 2.29, 1.78; in the third, 1.98, 1.85. That is more updates per epoch than batch gradient descent, and steadier votes than SGD.

12. Side by side

After one epoch	Batch	Mini-batch (B = 2)	Stochastic
votes (point gradients) computed	4	4	4
updates of w	1	2	4
w	1.5	2.1875	2.2292
J(w)	2.63	1.01	1.14
later behaviour	smooth; settles at 2	small wobble round 2	larger jitter (1.7 to 2.4)

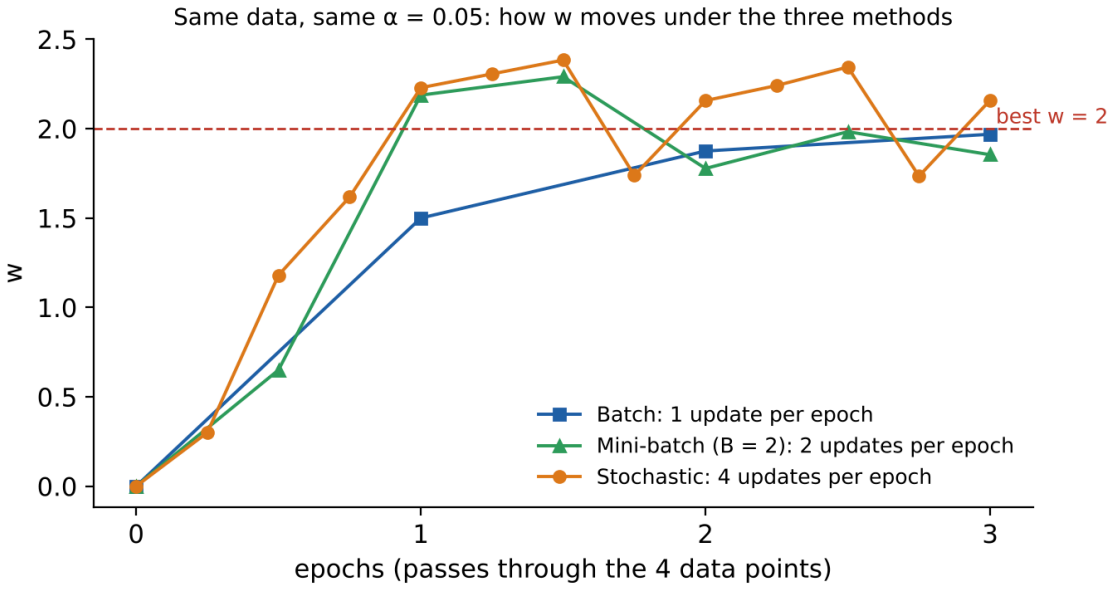


Figure 5. *w after every update, for three epochs. Each method does the same work per epoch; they differ in how many steps that work buys.*

In plain words: same work, more steps. Batch gradient descent takes one careful step per epoch; SGD takes many quick, rough steps. On large data sets the many rough steps win, because each careful step costs a full pass through the data. (Mini-batch beating SGD after epoch 1 here is luck of the order with only four points, not a rule.)

13. Why do noisy votes still work, and why the jitter?

Look at the votes exactly at the best value w = 2:

Point i	1	2	3	4	average
$g_i(2)$	−2	−4	6	0	0
the point's own best w (y_i/x_i)	3	2.5	1.67	2	2

The average vote is 0, as it must be at the bottom of J. The individual votes are not 0: points 1 and 2 would like w to be larger (their own best values are 3 and 2.5), point 3 would like it smaller (1.67) and point 4 is satisfied. SGD listens to one voter at a time, so even at the bottom it keeps being pushed around: that is the jitter.

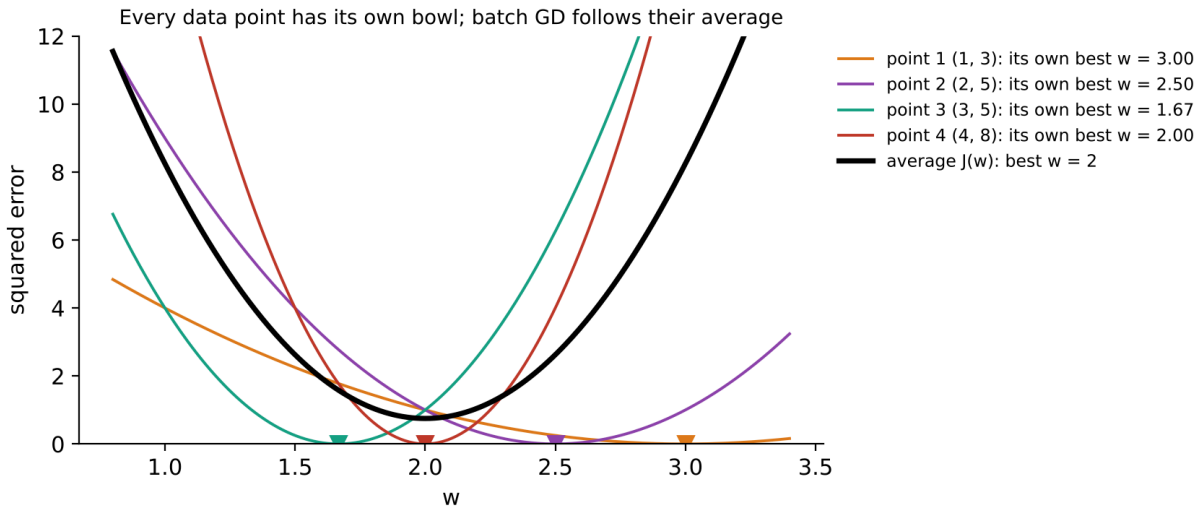


Figure 6. *Each point has its own bowl with its own bottom. Batch gradient descent follows the average bowl (black); SGD follows one coloured bowl per step.*

Now average two votes. The six possible mini-batches of size 2 give the averages −3, 2, −1, 1, −2 and 3. Their average is still 0, but they spread less (−3 to 3) than single votes (−4 to 6). Averaging more votes gives a steadier direction; averaging all of them gives the exact gradient.

> **Analogy:** an opinion poll. Asking one random person is quick but unreliable; asking 32 people is still quick and much more reliable; asking the whole country is exact but slow. On average all three point the right way, which is why SGD and mini-batch still get to the bottom region.

Two cures for the jitter: (i) **shrink α as training goes on** (a learning-rate schedule). With α = 0.05/epoch, SGD settles near 2 (w = 1.99 after 10 epochs) while the constant-α version keeps swinging (Figure 7). (ii) **Use larger mini-batches.** Also **shuffle** the data every epoch: if the data were sorted (all cats, then all dogs), each run of similar points would drag w the same way.

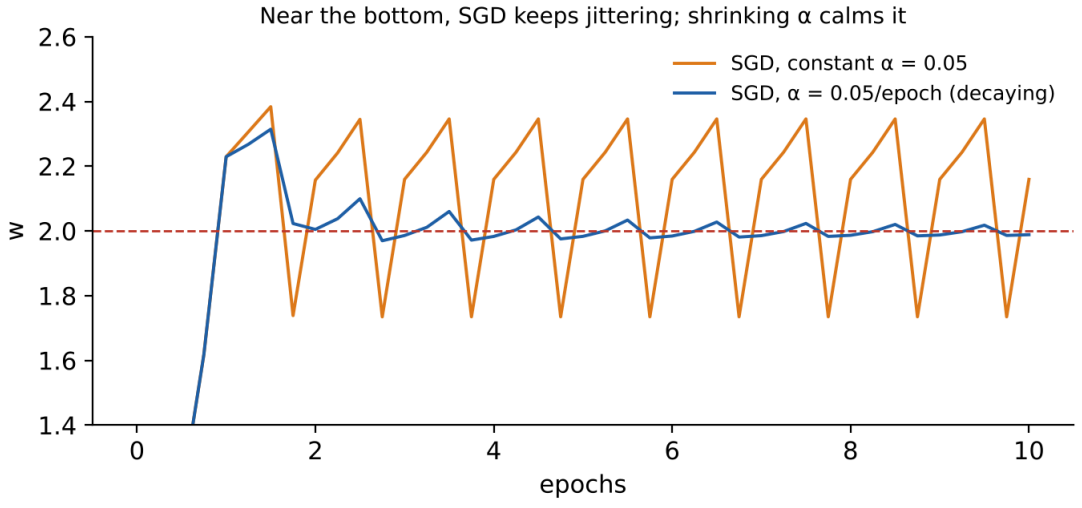


Figure 7. *SGD for ten epochs with a constant and with a decaying learning rate.*

14. The three methods compared

	Batch GD	Stochastic GD	Mini-batch GD
points per update	all n	1	B (typically 32 to 256)
updates per epoch	1	n	n/B

	Batch GD	Stochastic GD	Mini-batch GD
cost of one update	high: whole data set	very low	low
direction	exact gradient	very noisy	mildly noisy
path	smooth	zig-zag; jitters near the minimum	slight wobble
memory	whole data set	one point	one mini-batch
parallel hardware (GPU)	—	poor use	good use: B points at once
typical use	small data sets	streaming (online) data	almost all deep learning

Part C. The connection to machine learning and deep learning

15. What "training a model" means

Every supervised machine-learning model has adjustable numbers called **parameters** or **weights**. Training means choosing the weights that make the average loss over the training data as small as possible, and gradient descent, in one of its three forms, is how that minimisation is done. So Part B was not a toy version of training: it **was** training, with one weight and four data points.

Term in ML	Meaning	In our example
weight / parameter	a knob that the model learns	w
loss (cost) function	average error over the training data	J(w), the MSE
gradient	direction in which the loss increases fastest	J′(w)
learning rate	step size	$\alpha = 0.05$
epoch	one pass through the training data	all 4 points
batch size	points used per update	4 (batch), 1 (SGD), 2 (mini-batch)
iteration (step)	one update of the weights	1, 4 or 2 per epoch

16. Linear regression with two weights

Worked Example 7. Training $\hat{y} = wx + b$ on the same four points

Now the line need not pass through the origin. The loss is $J(w, b) = (1/n)\,\Sigma\,(wx_i + b - y_i)^2$, and with errors $e_i = \hat{y}_i - y_i$ the two slopes are averages of "error × input" and of "error":

$$\partial J/\partial w = (2/n)\,\Sigma\, e_i x_i, \qquad \partial J/\partial b = (2/n)\,\Sigma\, e_i$$

- **One batch step** from (w, b) = (0, 0) with $\alpha = 0.05$: the errors are −3, −5, −5, −8, so $\Sigma e_i x_i = -60$ and $\Sigma e_i = -21$. Then $\partial J/\partial w = -30$ and $\partial J/\partial b = -10.5$, and the new weights are (w, b) = (1.5, 0.525).
- **One SGD step** using point 1 only: $e_1 = -3$, so $\partial/\partial w = 2(-3)(1) = -6$ and $\partial/\partial b = 2(-3) = -6$; the new weights are (0.3, 0.3).
- Repeating either until the steps become tiny gives the least-squares line $\hat{y} = 1.5x + 1.5$, the same answer as the formula from statistics. Gradient descent is used instead of the formula when there are millions of data points or thousands of weights, where the formula becomes too expensive.

17. Logistic regression (classification): the same pattern

To classify, say spam (y = 1) or not spam (y = 0), logistic regression predicts a probability $\hat{y} = \sigma(wx + b)$, where $\sigma(z) = 1/(1 + e^{-z})$, and uses the cross-entropy loss. You may take its gradient for one example as given; it has the same "error × input" shape (without the 2):

$$\partial \ell / \partial w = (\hat{y} - y)\, x, \qquad \partial \ell / \partial b = (\hat{y} - y)$$

Worked Example 8. One SGD step for logistic regression

One email with feature x = 2 is spam (y = 1). Start with w = b = 0 and use $\alpha = 0.1$. Then z = 0 and $\hat{y} = \sigma(0) = 0.5$, so the error is $\hat{y} - y = -0.5$. The slopes are $\partial/\partial w = -0.5 \times 2 = -1$ and $\partial/\partial b = -0.5$, so $w = 0 - 0.1(-1) = 0.1$ and b = 0.05. The new prediction is $\sigma(0.1 \times 2 + 0.05) = \sigma(0.25) = 0.562$: it has moved towards the correct answer 1.

18. Neural networks and deep learning

A neural network is the same idea with many more weights (thousands to billions) arranged in layers. Only two things change.

- **How the gradient is computed.** It is found by **backpropagation**, which is the chain rule applied layer by layer and is done automatically by libraries such as PyTorch and TensorFlow. For each example it gives one vote per weight, exactly like g_i.
- **How much data is used per step.** Data sets are huge (millions of images), so the full gradient is never computed: training always uses mini-batches.

Every deep-learning training program has this shape:

```
for each epoch:
    shuffle the training data
    split it into mini-batches of size B
    for each mini-batch:
        forward pass : compute predictions ŷ for the B examples
        loss         : average the loss over the B examples
        backward pass: backpropagation computes the gradient
        update       : every weight  w ← w − α × (its gradient)
```

A note on names: in deep-learning libraries the optimiser called "SGD" (for example torch.optim.SGD) is almost always fed mini-batches. In practice "SGD" usually means **mini-batch** SGD.

Worked Example 9. Counting updates

The CIFAR-10 image data set has 50,000 training images. With mini-batches of B = 128, one epoch has $\lceil 50{,}000/128 \rceil = 391$ iterations (the last mini-batch has only 80 images), so 30 epochs make 391 × 30 = 11,730 weight updates. Batch gradient descent would make only 30 updates in those 30 epochs, and pure SGD would make 1,500,000.

Mini-batches suit GPUs: a GPU computes the predictions for 128 images in parallel, far faster than 128 separate one-image calculations. So the average of 128 votes costs little more than a single vote, and the step is much steadier.

Popular optimisers such as **momentum** and **Adam** are built on mini-batch SGD. They keep a running average (an exponentially weighted moving average, the same idea as exponential smoothing in Unit 3) of recent gradients, which smooths out the mini-batch noise.

19. Practical rules of thumb

- Start with mini-batch gradient descent with B = 32 to 256 (powers of 2 suit GPU memory).

- Try $\alpha = 0.1$, 0.01 and 0.001, and keep the largest value for which the loss goes down steadily.
- Shuffle the training data every epoch.
- Plot the loss after every epoch. A noisy but downward trend is normal for SGD and mini-batch; a rising loss means that α is too large.
- Reduce α later in training (for example, divide it by 10 when the loss stops improving) to calm the jitter.
- Larger B gives smoother steps, needs more memory and makes fewer updates per epoch; smaller B gives noisier steps and more updates.

20. Summary

1. Gradient descent repeats $x \leftarrow x - \alpha f'(x)$. The minus sign makes every step go downhill.
2. The learning rate α sets the step length: too small is slow, too large overshoots and can diverge.
3. In ML the loss is an average over the data, so its gradient is the **average of the per-example gradients** (the votes).
4. **Batch** counts all the votes per step (exact but slow per step); **SGD** counts one (fast but noisy); **mini-batch** counts B (the practical middle, and the standard in deep learning).
5. The noisy votes are right on average, which is why SGD works. They also cause jitter near the minimum, which is cured by a decaying α or larger mini-batches.
6. The same algorithm trains linear regression, logistic regression and deep neural networks. Only the way the gradient is computed changes (backpropagation in deep learning).

Practice problems (pen and paper)

1. $f(x) = (x + 2)^2$, $x_0 = 2$, $\alpha = 0.25$. Carry out three steps of gradient descent. By what factor does the distance to the minimum shrink at each step?
2. For $f(x) = 4x^2$, show that $x_{new} = (1 - 8\alpha)\, x_{old}$. For which α does gradient descent converge? Which α reaches the minimum in one step?
3. Data (1, 2), (2, 2), (3, 5); model $\hat{y} = wx$; loss MSE. (a) Setting $J'(w) = 0$ gives $w^* = \Sigma x_i y_i / \Sigma x_i^2$; find w^*. (b) Take one batch step from $w = 0$ with $\alpha = 0.05$. (c) Take one epoch of SGD in the order given, with $\alpha = 0.05$. (d) Take one epoch of mini-batch gradient descent with mini-batches {1, 2} and {3}, $\alpha = 0.05$. Compare the three results with w^*.
4. The MNIST data set has 60,000 training images. With B = 100, how many iterations are there per epoch, and how many updates in 10 epochs? How many updates would batch gradient descent and pure SGD make in 10 epochs?
5. One SGD step for logistic regression: $x = 1$, $y = 0$, $w = 0.5$, $b = 0$, $\alpha = 0.2$. Find $\hat{y}$, the error, the new w and b, and the new prediction. Did the prediction move the right way?
6. In Worked Example 5, explain in your own words why SGD with a constant α never settles exactly at $w = 2$. Give two ways to make it settle.
7. A friend says: "SGD is worse than batch gradient descent because its steps point in the wrong direction." How would you reply?

Answers

1. $f'(x) = 2(x + 2)$, so $x_{new} = x - 0.5(x + 2)$: $x = 0, -1, -1.5$. The distance to -2 halves every step (4, 2, 1, 0.5).

2. $f'(x) = 8x$ gives $x_{new} = x - 8\alpha x = (1 - 8\alpha)x$. It converges when $|1 - 8\alpha| < 1$, i.e. $0 < \alpha < 0.25$; $\alpha = 0.125$ reaches 0 in one step.
3. (a) $w^* = (2 + 4 + 15)/(1 + 4 + 9) = 21/14 = 1.5$ ($J = 0.5$). (b) $J'(0) = (2/3)(-21) = -14$, so $w = 0.7$ ($J = 3.49$). (c) Votes $-4 \rightarrow w = 0.2$; $-6.4 \rightarrow w = 0.52$; $-20.64 \rightarrow w = 1.552$ ($J = 0.513$). (d) Average of -4 and -8 is $-6 \rightarrow w = 0.3$; then $2 \cdot 3 \cdot (0.9 - 5) = -24.6 \rightarrow w = 1.53$ ($J = 0.504$). After one epoch SGD and mini-batch are almost at 1.5, while batch has reached only 0.7.
4. 600 iterations per epoch; 6,000 updates in 10 epochs. Batch: 10 updates. SGD: 600,000 updates.
5. $z = 0.5$, $\hat{y} = \sigma(0.5) = 0.622$, error 0.622. $w = 0.5 - 0.2(0.622)(1) = 0.376$, $b = -0.124$. The new prediction $\sigma(0.251) = 0.562$ is closer to the correct answer 0.
6. At $w = 2$ the average vote is zero, but the single votes are -2, -4, 6 and 0, so each SGD step still moves w. Cures: shrink α over the epochs, or average more votes per step (larger mini-batches).
7. Each SGD step is noisy, but on average it points in the correct direction, and it costs only one example instead of the whole data set. Per epoch SGD makes n steps instead of 1, so on large data sets it gets close to the minimum far sooner. A decaying α removes the final jitter.

Further reading

- MIT 6.036 Introduction to Machine Learning, lecture notes, chapter "Gradient Descent" (MITx Open Learning Library).
- Google Machine Learning Crash Course, "Linear regression: Hyperparameters" (learning rate, batch size, epochs).
- Goodfellow, Bengio and Courville, Deep Learning, MIT Press, 2016, Chapter 8, Section 8.1.3 "Batch and Minibatch Algorithms" and Section 8.3.1 "Stochastic Gradient Descent" (deeplearningbook.org).
- Deisenroth, Faisal and Ong, Mathematics for Machine Learning, Cambridge University Press, 2020, Chapter 7 (syllabus reference; mml-book.github.io).
- Kingma and Ba, Adam: A Method for Stochastic Optimization, ICLR 2015 (arXiv 1412.6980).